

Frameworks, architectures & API

Comprendre ce qu'on vous demandera d'écrire, avant d'ouvrir Spring Boot.

AU PROGRAMME AUJOURD'HUI

1. Ce qu'est un framework, et en quoi il diffère d'une bibliothèque
2. L'inversion de contrôle : le principe qui change tout
3. Les principes de conception qui justifient une architecture
4. L'architecture en couches : présentation, métier, persistance
5. Les patrons de présentation : MVC, MVP, MVVM
6. Les architectures d'ensemble : monolithe, microservices, hexagonale
7. **Ce qu'est une API**, et ce qu'est une **API REST**
8. **Le flux CRUD complet**, du clic jusqu'à la table PostgreSQL
9. Deux ponts vers Spring Boot : injection de dépendances et ORM

PRÉREQUIS

- **Modélisation** — MCD, MLD, MPD : entités, associations, cardinalités, clés étrangères
- **SQL / PostgreSQL** — `CREATE TABLE`, `SELECT`, `JOIN`, contraintes
- **POO Java** — classes, objets, attributs, méthodes, constructeurs, interfaces, héritage

Aucune ligne de Spring n'est exigée aujourd'hui. On construit le vocabulaire et les schémas mentaux.

Pourquoi cette séance

Le chaînon manquant entre ce que vous savez déjà et ce qui vous attend.

Vous savez aujourd'hui faire trois choses, et vous les faites séparément. Vous savez modéliser un domaine et le traduire en tables PostgreSQL. Vous savez interroger ces tables en SQL. Vous savez écrire des classes Java qui manipulent des objets.

Ce que personne ne vous a encore expliqué, c'est **comment on assemble ces trois compétences dans une seule application**, et pourquoi tout le monde les assemble à peu près de la même manière. Cette manière porte un nom : une **architecture**. L'outil qui vous la propose — ou vous l'impose — porte un nom : un **framework**. Et la porte d'entrée par laquelle le monde extérieur s'adresse à votre application porte un nom : une **API**.

L'OBJECTIF DU JOUR

À la fin de cette séance, vous devez pouvoir lire une offre d'emploi qui dit « architecture n-tiers, Spring MVC, couche service, repository JPA, API REST » et comprendre chaque mot. Pas savoir l'écrire — le *comprendre*.

Bibliothèque ou framework ?

La différence n'est pas une question de taille. C'est une question de qui commande.

2.1 La boîte à outils et la maison préfabriquée



MÉTAPHORE DU CHANTIER

Vous devez construire une maison.

Première option : la boîte à outils. On vous livre un marteau, une scie, une perceuse.

Excellents outils. Mais vous décidez de tout : où poser les fondations, quand monter les murs, dans quel ordre. Vous prenez le marteau quand *vous* en avez besoin, et vous le reposez. Vous êtes le chef de chantier.

Seconde option : la maison préfabriquée. On vous livre une structure déjà debout : murs porteurs, dalle, gaines électriques, arrivées d'eau. Vous n'avez plus qu'à aménager l'intérieur, aux emplacements prévus. Vous gagnez six mois. Mais vous ne mettez pas la salle de bain où vous voulez : les arrivées d'eau sont déjà là.

La boîte à outils, c'est une **bibliothèque**. La maison préfabriquée, c'est un **framework**.

⇔ faites glisser

BIBLIOTHÈQUE



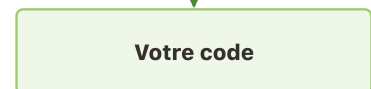
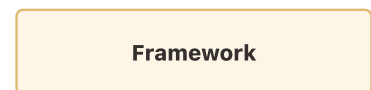
vous appelez
quand vous voulez



Vous gardez le contrôle du déroulement.

il vous appelle
quand il l'a décidé

FRAMEWORK



Le framework garde le contrôle.

Figure 1 — Le seul critère qui distingue une bibliothèque d'un framework : le sens de la flèche.

Une **bibliothèque** (*library*) est un ensemble de classes et de fonctions déjà écrites, que **vous appelez** quand vous en avez besoin. Vous gardez la main sur le déroulement du programme.

Un **framework** — le mot se traduit par « cadre de travail », « ossature » — est un squelette d'application déjà construit, qui **appelle votre code** aux moments qu'il a prévus. C'est lui qui garde la main.

Le même travail, vu des deux côtés

JAVA

```
// — BIBLIOTHÈQUE : c'est MOI qui appelle —————  
public class Programme {  
    public static void main(String[] args) {  
        Client client = new Client("Dupont");    // mon code  
        String json = new Gson().toJson(client);    // j'appelle la biblio  
        System.out.println(json);                // je reprends la main  
    }  
}
```

Gson est une bibliothèque de conversion en JSON. Je l'appelle, elle répond, je continue.

JAVA

```
// — FRAMEWORK : c'est LUI qui m'appelle —————  
@RestController  
public class ClientController {  
  
    @GetMapping("/clients")                // « appelle-moi sur cette URL »  
    public List<Client> lister() {          // je n'appelle JAMAIS cette méthode  
        return service.tousLesClients();  
    }  
}
```

Extrait Spring, donné ici uniquement pour l'illustration. On y reviendra.

Analysons ce second bloc ligne par ligne :

LIGNE	CE QU'ELLE FAIT RÉELLEMENT
<code>@RestController</code>	Une annotation : une étiquette collée sur la classe. Elle ne fait rien toute seule. Elle dit au framework : « cette classe contient des points d'entrée web, viens la chercher au démarrage ».
<code>@GetMapping("/clients")</code>	Une seconde étiquette : « quand une requête HTTP GET arrive sur l'adresse <code>/clients</code> , c'est cette méthode qu'il faut exécuter ».
<code>public List<Client> lister()</code>	Ma méthode. Nulle part dans mon projet je n'écris <code>lister()</code> pour l'exécuter. C'est le framework qui le fera, quand un navigateur se connectera.
<code>return service...</code>	Je renvoie une liste d'objets Java. Le framework se chargera seul de la transformer en JSON et de l'envoyer sur le réseau.

	BIBLIOTHÈQUE	FRAMEWORK
Qui appelle qui	Votre code appelle la bibliothèque	Le framework appelle votre code
Qui écrit le <code>main</code>	Vous	Le framework (vous ne faites que le démarrer)
Structure du projet	Libre	Imposée, ou fortement conseillée
On l'enlève ?	Facile : quelques appels à réécrire	Difficile : c'est la structure porteuse
Exemples Java	Gson, Apache Commons, JUnit	Spring Boot, Jakarta EE, Quarkus

⚠ LE PIÈGE DU VOCABULAIRE

« Framework » n'est pas un synonyme de « grosse bibliothèque ». Un framework peut être petit, une bibliothèque peut être énorme. Le seul critère qui tranche est celui du **sens de l'appel** : si c'est vous qui appelez, c'est une bibliothèque ; si c'est lui qui vous appelle, c'est un framework.

2.2 L'inversion de contrôle



LE CHANTIER, SUITE

Sur un chantier ordinaire, le plombier arrive quand il veut, pose ses tuyaux, s'en va. Résultat : il passe avant le maçon, le maçon casse ses tuyaux, tout est à refaire.

Sur un chantier organisé, il y a un **chef de chantier**. Le plombier lui laisse son numéro et rentre chez lui. Le chef l'appelle le jour où c'est son tour. Le plombier a perdu la liberté de décider *quand* il travaille. Il a gagné une maison qui tient debout.

Ce renversement porte un nom : l'**inversion de contrôle** (*Inversion of Control, IoC*). Dans un programme classique, votre code décide de l'enchaînement et appelle les outils. Avec un framework, l'enchaînement appartient au framework, et c'est lui qui vient chercher vos morceaux de code.

On résume ce principe par une formule empruntée aux castings de cinéma, le **principe d'Hollywood** : « *Ne nous appelez pas, nous vous appellerons.* » Vous laissez vos coordonnées — vos annotations — et vous attendez.

↔ faites glisser

FOURNI PAR LE FRAMEWORK · boucle principale · serveur HTTP · connexion base · sécurité

EMPLACEMENT 1

Votre contrôleur

reçoit et renvoie

EMPLACEMENT 2

Votre service

applique les règles

EMPLACEMENT 3

Votre repository

lit et écrit

Vous n'écrivez que le contenu des trois emplacements. Le reste existe déjà — et c'est lui qui déclenche votre code.

Figure 2 — L'inversion de contrôle vue de l'extérieur : le framework est le contenant, votre code est le contenu.

⚠ IOC ET INJECTION DE DÉPENDANCES NE SONT PAS SYNONYMES

L'**inversion de contrôle** est un *principe* général : le contrôle du flux passe à un cadre extérieur. L'**injection de dépendances** (*Dependency Injection, DI*) n'en est qu'une *technique* de mise en œuvre parmi d'autres. Tout DI est de l'IoC ; tout IoC n'est pas du DI. Confondre les deux est l'erreur la plus fréquente en entretien d'embauche.

2.3 Ce qu'un framework vous donne, ce qu'il vous prend

Un framework n'est pas gratuit. Le choisir est un arbitrage, pas un réflexe.

CE QU'IL VOUS DONNE	CE QU'IL VOUS PREND
Du temps. Connexion à la base, sécurité, gestion des erreurs, serveur web : tout est déjà écrit.	Du temps, d'abord. Apprendre Spring prend des semaines. Sur un script de 200 lignes, vous perdez au change.
Des conventions. Un développeur qui arrive sur votre projet retrouve ses repères en une heure.	De la liberté. On fait les choses à la manière du framework, ou on souffre.
Du code éprouvé. Des milliers de projets ont déjà trouvé les bugs à votre place.	De la transparence. Beaucoup de choses se passent « toutes seules ». Quand ça casse, il faut savoir pourquoi.
Un écosystème. Documentation, forums, modules, recrutement.	De l'indépendance. C'est l' <i>enfermement propriétaire (vendor lock-in)</i> : en sortir coûte cher.

💡 LA RÈGLE DE DÉCISION

Un framework est rentable quand le projet est **durable**, **collectif** et **standard**. Une application de gestion sur trois ans, à cinq développeurs : oui. Un script d'import lancé une fois : non, une bibliothèque suffit.

SECTION 03

Les principes de conception

Une architecture n'est pas un goût personnel. C'est la réponse à quatre ou cinq principes.

3.1 La séparation des responsabilités



MÉTAPHORE DU RESTAURANT — ON LA GARDE JUSQU'À LA FIN

Dans un restaurant qui tourne, personne ne fait tout. Le serveur prend les commandes et apporte les assiettes ; il ne cuisine pas. Le cuisinier prépare ; il ne va jamais en salle. Le magasinier gère le stock ; il ne touche ni aux clients ni aux casseroles.

Imaginez maintenant l'inverse : un seul employé qui prend la commande, court en cuisine, redescend en réserve, remonte, ressert. Aux heures creuses, ça marche. À midi, tout s'effondre. Et le jour où il tombe malade, le restaurant ferme.

La **séparation des responsabilités** (*separation of concerns*) consiste à découper un programme en parties qui ont chacune **un seul métier**. Une partie affiche. Une partie décide. Une partie stocke. Aucune ne fait le travail d'une autre.

Voici ce que donne le non-respect de ce principe. Ce code fonctionne, et c'est bien le problème :

JAVA

```
// ✗ Une seule méthode qui fait TOUT : afficher, décider, stocker
public void inscrire(String nom, int age) {
    if (age < 18) {                                // règle métier
        System.out.println("Refusé : mineur");    // affichage
        return;
    }
    Connection c = DriverManager.getConnection(...); // accès base
    c.prepareStatement("INSERT INTO client ...").execute();
    System.out.println("Bienvenue " + nom);        // affichage
}
```

Analysons ce code ligne par ligne :

- `if (age < 18)` — une **règle métier**. Elle appartient au domaine, pas à l'écran.
- `System.out.println` — de l'**affichage**, en console. Le jour où l'application devient un site web, cette ligne ne sert plus à rien.
- `DriverManager.getConnection` — de l'**accès aux données**, écrit en dur. Changer de base oblige à rouvrir cette méthode.

Trois métiers en huit lignes. Conséquence concrète : cette règle des 18 ans est **intestable** sans base de données allumée, et **irrutilisable** ailleurs.

3.2 Couplage et cohésion



LA GUIRLANDE ET LE TIROIR

Le couplage, c'est la vieille guirlande de Noël montée en série : une ampoule grille, tout s'éteint, et vous passez la soirée à chercher laquelle. Les guirlandes modernes sont en parallèle : une ampoule grille, les autres continuent.

La cohésion, c'est le tiroir à couverts : fourchettes avec fourchettes, couteaux avec couteaux. Son contraire est le tiroir fourre-tout de la cuisine, où cohabitent des piles, du scotch, une clé sans serrure et trois vis.

Le **couplage** mesure à quel point deux parties du code dépendent l'une de l'autre. La **cohésion** mesure à quel point les éléments d'une même partie vont réellement ensemble.



LA FORMULE À RETENIR

Couplage faible, cohésion forte. Chaque module s'occupe d'une seule chose (cohésion forte) et dépend le moins possible des autres (couplage faible). Toute l'architecture logicielle tient dans ces quatre mots.

3.3 DRY, KISS, YAGNI

SIGLE	CE QU'IL DIT	L'IMAGE
DRY <i>Don't Repeat Yourself</i>	Ne vous répétez pas. Chaque règle est écrite à un seul endroit.	La recette est notée une fois dans le carnet, pas recopiée sur douze bouts de papier — sinon, le jour où vous changez la dose de sel, vous en oubliez huit.
KISS <i>Keep It Simple, Stupid</i>	Faites simple. La solution la plus courte qui marche est presque toujours la bonne.	Pour ouvrir une boîte de conserve, un ouvre-boîte. Pas une machine à quinze engrenages qui fait aussi le café.
YAGNI <i>You Aren't Gonna Need It</i>	Vous n'en aurez pas besoin. N'écrivez pas aujourd'hui ce qui « servira peut-être un jour ».	Emporter une tente et un réchaud pour aller au bureau, au cas où.

⚠ DRY MAL COMPRIS FAIT PLUS DE MAL QUE BIEN

Deux morceaux de code qui se *ressemblent* ne sont pas forcément des doublons. DRY parle de **connaissance** dupliquée, pas de caractères dupliqués. Le calcul d'une TVA et celui d'une remise peuvent s'écrire pareil : les fusionner en une seule méthode vous garantit une catastrophe le jour où la TVA change.

3.4 SOLID en cinq phrases

SOLID est un moyen mnémotechnique pour cinq principes de conception orientée objet. Deux d'entre eux vous serviront dès la semaine prochaine ; retenez surtout ceux-là.

LETTRE	PRINCIPE	EN UNE PHRASE
S	Responsabilité unique <i>Single Responsibility</i>	Une classe ne doit avoir qu' une seule raison de changer .
O	Ouvert / fermé <i>Open/Closed</i>	On doit pouvoir ajouter un comportement sans rouvrir le code existant.
L	Substitution de Liskov	Une classe fille doit pouvoir remplacer sa classe mère sans rien casser.
I	Ségrégation des interfaces	Mieux vaut plusieurs petites interfaces qu'une grosse fourre-tout.
D	Inversion des dépendances <i>Dependency Inversion</i>	On dépend d' interfaces , jamais de classes concrètes.

Le « D », en détail — c'est lui qui prépare Spring



LA PRISE ÉLECTRIQUE

Votre lampe de bureau ne connaît pas la centrale nucléaire. Elle ne connaît qu'une chose : une prise à deux trous, 230 volts. C'est un **contrat standard**. Derrière la prise, on peut passer du nucléaire à l'éolien sans que la lampe s'en aperçoive — et vous pouvez brancher n'importe quelle lampe.

La prise, c'est l'**interface**. La lampe ne dépend pas d'un producteur d'électricité : elle dépend d'une prise.

```
// ❌ COUPLAGE FORT – le service fabrique lui-même sa dépendance
public class ServiceClient {
    private ClientDaoPostgres dao = new ClientDaoPostgres();
    //      ↑ soudé à PostgreSQL, pour toujours
}

// ✅ INVERSION DES DÉPENDANCES – on dépend d'un contrat
public interface ClientDao {                // la « prise »
    Client trouverParId(Long id);
}

public class ServiceClient {
    private final ClientDao dao;                // une interface, pas une classe

    public ServiceClient(ClientDao dao) {        // on ME donne l'implémentation
        this.dao = dao;
    }
}
```

Analysons ce code ligne par ligne :

1. `new ClientDaoPostgres()` — la classe fabrique elle-même son outil. Elle est **soudée** à PostgreSQL : impossible de la tester sans base réelle, impossible de changer de stockage.
2. `interface ClientDao` — on déclare le **contrat** : « quelqu'un, quelque part, sait retrouver un client par son identifiant ». On ne dit pas comment.
3. `private final ClientDao dao` — le service ne connaît plus que le contrat. `final` garantit que la dépendance ne changera pas en cours de route.
4. `public ServiceClient(ClientDao dao)` — la dépendance **entre par le constructeur**. Le service ne la fabrique plus : on la lui fournit. C'est exactement ce que Spring automatisera.

💡 LE TEST QUI NE MENT PAS

Posez-vous cette question devant n'importe quelle classe : « **puis-je la tester sans allumer la base de données ?** » Si la réponse est non, le couplage est trop fort. Avec la seconde version, il suffit de passer une fausse implémentation de `ClientDao` qui renvoie un client inventé.

L'architecture en couches

Le découpage que vous retrouverez dans neuf applications de gestion sur dix.

4.1 Les trois étages du restaurant



LE RESTAURANT, SUITE

Trois espaces, trois métiers, une circulation stricte :

La salle — le serveur accueille, prend la commande, apporte l'assiette. Il ne décide pas de la recette et ne descend jamais en réserve.

La cuisine — on applique les recettes, on refuse un plat si un ingrédient manque, on décide qu'un plat n'est pas servi le dimanche. C'est là que sont les *règles*.

La réserve — on range, on retrouve, on sort les ingrédients. Aucune décision : on stocke et on rend ce qu'on demande.

Et surtout : **le client ne va jamais en réserve**. Il parle au serveur, qui parle à la cuisine, qui parle à la réserve. Chacun ne s'adresse qu'à son voisin immédiat.

↔ faites glisser

AU RESTAURANT

CE QU'ELLE NE FAIT JAMAIS

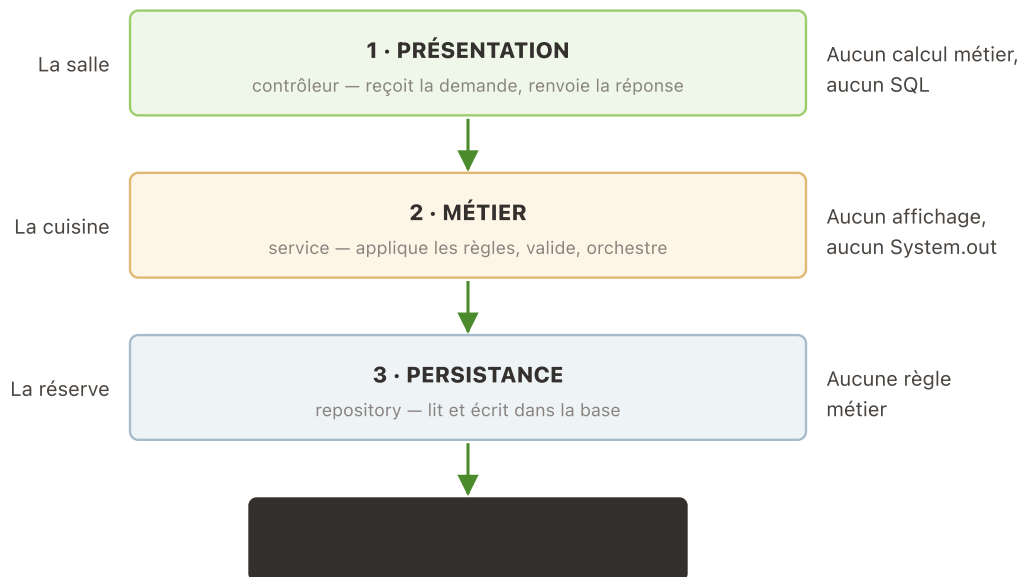


Figure 3 — Chaque couche ne s'adresse qu'à celle immédiatement en dessous. Un contrôleur qui écrit du SQL saute deux étages : c'est une faute d'architecture.

🔗 LA RÈGLE DE CIRCULATION

Chaque couche ne parle qu'à **celle du dessous**. La présentation appelle le métier, le métier appelle la persistance. Jamais l'inverse, et jamais de saut : un contrôleur qui écrit du SQL est une faute d'architecture, même si le code fonctionne.

4.2 Le vocabulaire des couches

Ces cinq mots reviendront chaque jour à partir de la semaine prochaine. Ce sont des **rôles**, pas des mots-clés du langage : ce sont de simples classes Java.

TERME	DÉFINITION	COUCHE	AU RESTAURANT
Contrôleur <i>Controller</i>	Reçoit la requête, appelle le service, renvoie une réponse	Présentation	Le serveur
Service	Contient les règles métier et enchaîne les opérations	Métier	Le chef de cuisine
Repository / DAO <i>Data Access Object</i>	Sait lire et écrire une table. Rien d'autre.	Persistance	Le magasinier
Entité <i>Entity</i>	Une classe Java qui reflète une table de la base	Traverse tout	L'ingrédient tel qu'il est stocké
DTO <i>Data Transfer Object</i>	Un objet fait pour sortir de l'application : uniquement les champs qu'on accepte de montrer	Présentation	L'assiette dressée qui part en salle

```

// Le même besoin, traversant les trois couches

// — PRÉSENTATION : je reçois, je délègue, je renvoie —
public class ClientController {
    private final ClientService service;

    public ClientDto fiche(Long id) {
        Client client = service.trouverParId(id);    // je délègue
        return ClientDto.depuis(client);           // je masque les champs privés
    }
}

// — MÉTIER : je décide —
public class ClientService {
    private final ClientRepository repository;

    public Client trouverParId(Long id) {
        Client c = repository.findById(id);
        if (c == null) throw new ClientIntrouvable(id); // règle métier
        return c;
    }
}

// — PERSISTANCE : je range et je retrouve —
public interface ClientRepository {
    Client findById(Long id);
    void save(Client client);
}

```

Analysons ce code ligne par ligne :

1. Le **contrôleur** ne contient aucune décision. Il traduit une demande extérieure en appel de service, puis traduit le résultat en objet de sortie.
2. Le **service** décide : ici, « un client introuvable est une erreur ». C'est une règle du domaine, elle vit donc au milieu.
3. Le **repository** est une *interface* — un contrat. On applique le « D » de SOLID : le service dépend d'un contrat, pas d'une implémentation PostgreSQL.
4. Chaque couche ne connaît que la suivante. Le contrôleur ignore jusqu'à l'existence du repository.

4.3 Où atterrissent vos MCD, MLD et MPD

RAPPEL DES SÉANCES DE MODÉLISATION

Vous avez descendu trois marches : le **MCD** décrit le monde réel (entités, associations, cardinalités) ; le **MLD** le traduit en tables et clés étrangères ; le **MPD** le concrétise dans un SGBD précis (types PostgreSQL, index, contraintes).

Il vous manquait la quatrième marche : **comment cette table redevient un objet Java.**

SQL

```
-- MPD : la table telle qu'elle existe dans PostgreSQL
CREATE TABLE client (
  id          SERIAL PRIMARY KEY,
  nom         VARCHAR(50) NOT NULL,
  email       VARCHAR(120) UNIQUE,
  date_inscription DATE NOT NULL
);
```

JAVA

```
// La même réalité, vue depuis Java : une classe « entité »
public class Client {
  private Long id;           // ↔ colonne id
  private String nom;        // ↔ colonne nom
  private String email;      // ↔ colonne email
  private LocalDate dateInscription; // ↔ colonne date_inscription
  // + constructeur, getters, setters
}
```

MONDE DES BASES DE DONNÉES	MONDE JAVA	CORRESPONDANCE
Table <code>client</code>	Classe <code>Client</code>	Une table ↔ une classe
Colonne <code>nom</code>	Attribut <code>nom</code>	Une colonne ↔ un attribut
Ligne (enregistrement)	Instance (objet)	Une ligne ↔ un objet
Clé primaire <code>id</code>	Attribut <code>id</code>	Identité de l'objet
Clé étrangère <code>id_commande</code>	Référence <code>Commande</code> <code>commande</code>	Une jointure ↔ une référence d'objet
<code>date_inscription</code> (snake_case)	<code>dateInscription</code> (camelCase)	Conventions de nommage différentes

Écrire cette traduction à la main, table par table, colonne par colonne, est fastidieux et répétitif. C'est exactement le travail que fera l'**ORM** — voir section 10.

⚠ UNE ENTITÉ N'EST PAS UN DTO

L'**entité** est le reflet fidèle de la table : elle contient *tout*, y compris le mot de passe haché, la note interne du commercial, l'identifiant technique. Le **DTO** est ce qu'on accepte de montrer à l'extérieur.

Renvoyer directement une entité dans une réponse web est la fuite de données la plus banale du métier. En réserve, il y a le carton entier ; en salle, on apporte une assiette.

💡 RECONNAÎTRE UNE BONNE COUCHE MÉTIER

Ouvrez une classe de service et cherchez les mots `SELECT`, `HttpServletRequest`, `System.out`. Si vous en trouvez, une couche déborde sur une autre. Une couche métier propre ne parle que le langage du domaine : clients, commandes, remises, factures.

MVC, MVP, MVVM

Trois manières d'organiser l'écran. Ce ne sont pas des architectures rivales : ce sont trois façons de découper la couche de présentation.

⚠ LA CONFUSION À DISSIPER TOUT DE SUITE

MVC n'est **pas** une alternative à l'architecture en couches. MVC organise la *couche présentation* ; les couches organisent l'*application entière*. Une application Spring normale est en trois couches **et** en MVC. Les deux cohabitent, ils ne se remplacent pas.

5.1 MVC — Modèle, Vue, Contrôleur



LE SERVICE EN SALLE

Vous commandez. Le **serveur** note, porte le bon en cuisine, revient avec l'assiette et la pose sur la **table**. La **cuisine** a préparé le plat sans jamais vous voir. La table présente le plat sans rien savoir de la recette.

Le serveur est le **contrôleur**, la cuisine est le **modèle**, la table est la **vue**.

↔ faites glisser

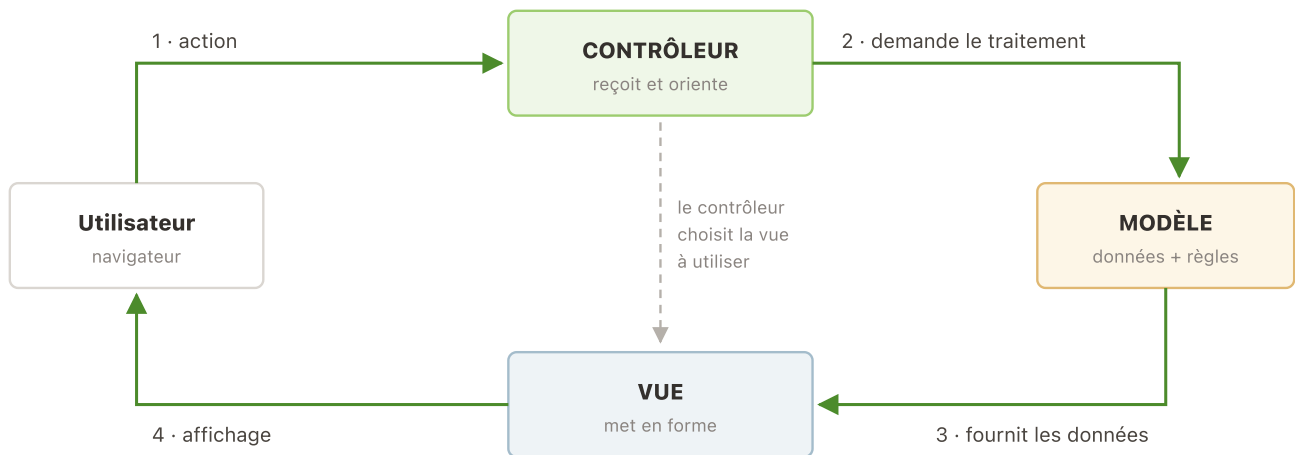


Figure 4 — Le cycle complet. Le contrôleur ne calcule rien et n'affiche rien : il oriente. Le modèle ignore qu'un écran existe.

ÉLÉMENT	RESPONSABILITÉ	CE QU'IL IGNORE
Modèle	Les données et les règles métier	Il ignore totalement l'existence d'un écran
Vue	L'affichage. Elle lit le modèle et le met en forme	Elle ne contient aucun calcul, aucune décision
Contrôleur	Recevoir l'action, appeler le modèle, choisir la vue	Il n'affiche rien et ne calcule rien

JAVA

```
// LE CONTRÔLEUR – il n'affiche rien, il ne calcule rien
public class ClientController {

    private final ClientService service; // le lien vers la cuisine

    public String afficherFiche(Long id, Modele modele) {
        Client client = service.trouverParId(id); // 1. je passe la commande
        modele.ajouter("client", client);         // 2. je pose le plat sur le passe
        return "fiche-client";                    // 3. je désigne la table
    }
}
```

Analysons ce code ligne par ligne :

1. `service.trouverParId(id)` — le contrôleur **délègue**. Il ne sait pas comment on trouve un client, il sait à qui demander.
2. `modele.ajouter("client", client)` — il dépose le résultat dans un sac de données que la vue viendra ouvrir. C'est le passe-plat.
3. `return "fiche-client"` — il renvoie un simple texte : le **nom** de la vue à utiliser. Il ne génère pas une ligne de HTML lui-même.

⚠ LE « M » DE MVC N'EST PAS LA BASE DE DONNÉES

Le modèle, ce sont les **données et les règles** du domaine, pas les tables PostgreSQL.

Dans une application en couches, le « M » de MVC recouvre en pratique la couche métier et la couche persistance réunies, vues depuis la présentation.

5.2 MVP — Modèle, Vue, Présentateur



LE SERVEUR QUI DRESSE À TABLE

Variante du service : le serveur ne se contente plus de poser l'assiette. Il apporte les éléments et **dresse le plat devant vous**, morceau par morceau. La table, elle, ne fait strictement rien : elle subit.

Dans MVP, la vue devient **passive**. Elle n'a plus le droit de lire le modèle. Le **présentateur** lui dicte tout, champ par champ : « écris *Dupont* dans le libellé du nom ».

```
// La VUE est réduite à un contrat d'affichage pur
public interface VueFicheClient {
    void afficherNom(String nom);
    void afficherErreur(String message);
}

// LE PRÉSENTATEUR pilote la vue instruction par instruction
public class PresentateurClient {
    private final VueFicheClient vue;      // il tient la vue
    private final ClientService service;

    public void charger(Long id) {
        try {
            Client c = service.trouverParId(id);
            vue.afficherNom(c.getNom());    // il DICTE l'affichage
        } catch (ClientIntrouvable e) {
            vue.afficherErreur("Client inconnu");
        }
    }
}
```

L'intérêt est le test : la vue étant une simple interface, on la remplace par une fausse implémentation et on vérifie que le présentateur a bien appelé `afficherErreur`. Aucun écran nécessaire.

5.3 MVVM et la liaison de données



LE PASSE-PLAT AUTOMATIQUE

Dernière évolution du restaurant : on installe un tapis roulant entre la cuisine et la table. Dès qu'un plat est prêt, il **apparaît** devant vous. Personne ne le porte. Et dans l'autre sens : dès que vous posez votre carte de fidélité sur la table, la cuisine le sait immédiatement.

Plus personne ne fait le trajet. Le tapis est branché une fois pour toutes.

Ce tapis roulant s'appelle la **liaison de données** (*data binding*), et c'est la seule vraie différence entre MVVM et les deux précédents.

Le **ViewModel** — « modèle de vue » — expose l'état de l'écran et les données mises en forme. Il ne connaît pas la vue et ne l'appelle jamais. La vue se *branche* dessus.

VUE · TEMPLATE

```
<!-- LA VUE : aucune logique, uniquement des branchements -->
<input v-model="nom">                                <!-- liaison à double sens -->
<p>Bonjour {{ nomEnMajuscules }}</p> <!-- liaison en lecture -->
```

JAVASCRIPT

```
// LE VIEWMODEL : l'état de l'écran, sans aucune référence à la vue
data() {
  return { nom: '' };                                // l'état
},
computed: {
  nomEnMajuscules() {
    return this.nom.toUpperCase();                    // recalculé tout seul
  }
}
```

Analysons ce code ligne par ligne :

1. `v-model="nom"` — la **double liaison** : l'utilisateur tape, l'attribut `nom` change instantanément ; on modifie `nom` par programme, le champ se met à jour. Aucun code pour ça.
2. `{{ nomEnMajuscules }}` — la vue affiche une valeur calculée sans savoir comment elle est calculée.
3. `nomEnMajuscules()` — cette fonction est **rejouée automatiquement** à chaque changement de `nom`. Personne ne l'appelle : le framework observe.
4. Nulle part le ViewModel ne dit `vue.afficher(...)`. Il ignore qu'une vue existe. C'est là toute la différence avec MVP.

↔ faites glisser

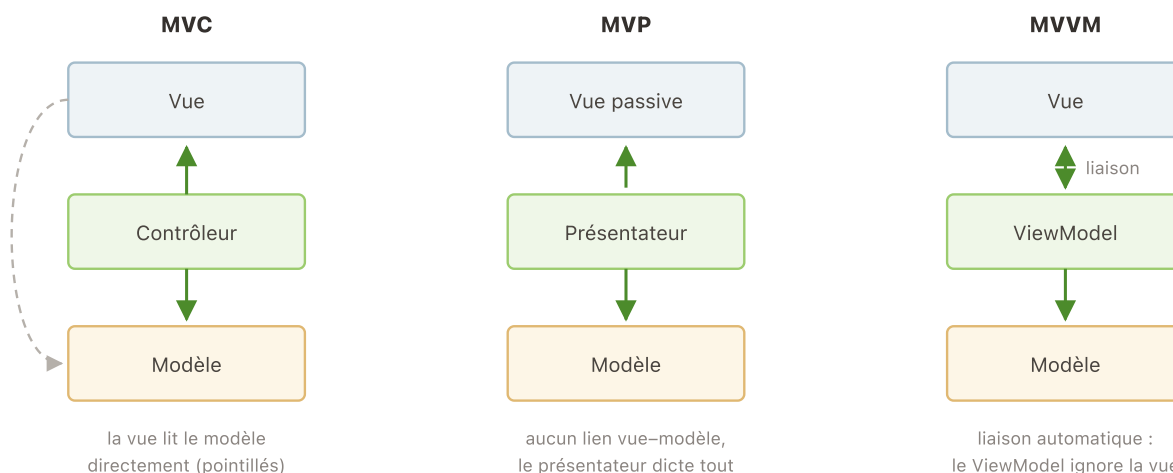


Figure 5 — Les trois patrons diffèrent sur un seul point : la nature du lien entre la vue et ce qui la nourrit.

⚠ LE VRAI CRITÈRE DE DISTINCTION

On lit souvent « MVC pour le web, MVVM pour le desktop ». C'est faux et ça ne vous aidera pas. Le critère réel est unique : **y a-t-il une liaison de données automatique ?** Si le framework observe les changements et rafraîchit l'écran seul, vous êtes en MVVM. Sinon, vous êtes en MVC ou MVP, quel que soit le support.

5.4 Comparatif

	MVC	MVP	MVVM
Le troisième rôle	Contrôleur	Présentateur	ViewModel
La vue lit-elle le modèle ?	Oui, directement	Non, jamais	Non, elle se lie au ViewModel
Le troisième rôle connaît-il la vue ?	Il choisit laquelle afficher	Oui, il la pilote	Non, il l'ignore totalement
Liaison automatique	Non	Non	Oui — c'est sa raison d'être
Rapport vue / rôle	1 contrôleur pour N vues	1 présentateur pour 1 vue	1 ViewModel pour N vues
Terrain habituel	Web côté serveur (Spring MVC, Symfony)	Anciennes applis Android, WinForms	Vue, Angular, WPF, Android moderne
Testabilité	Correcte	Très bonne	Très bonne

🎯 CE QU'ILS ONT TOUS EN COMMUN

Les trois patrons répondent à la même question : **comment empêcher que la logique métier se retrouve mélangée au code d'affichage ?** Ils diffèrent seulement sur la façon dont la vue et les données communiquent. Retenez le problème, les solutions se déduisent.

💡 VOUS NE CHOISIREZ PRESQUE JAMAIS

En pratique, le patron vous est imposé par l'outil : Spring MVC vous met en MVC, Vue et Angular vous mettent en MVVM. L'enjeu n'est donc pas de choisir, mais de **reconnaître dans quel patron vous êtes** pour savoir où poser chaque ligne de code.

Les architectures d'ensemble

On change d'échelle : non plus l'organisation du code, mais celle de l'application entière.

6.1 Monolithe, monolithe modulaire, microservices



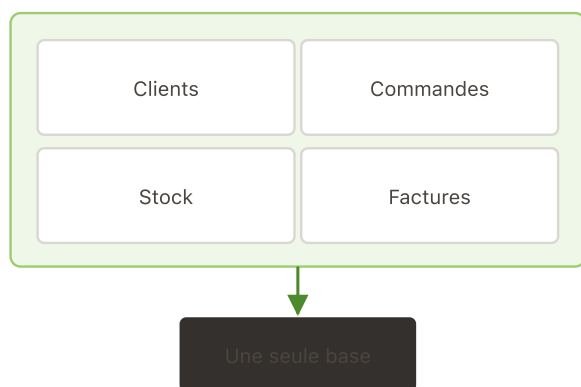
LE GRAND MAGASIN ET LA RUE COMMERÇANTE

Le grand magasin : tout sous un seul toit, une seule caisse, un seul système d'alarme. Simple, efficace, et l'on circule d'un rayon à l'autre sans sortir. Mais pour repeindre le rayon jouets, on ferme tout le magasin. Et si le courant saute, plus rien ne fonctionne.

La rue commerçante : boulangerie, boucherie, librairie. Chacune son local, sa caisse, ses horaires. La boulangerie ferme pour travaux, les autres continuent. Mais désormais, pour un repas complet, il faut faire quatre boutiques sous la pluie — et si la boucherie déménage sans prévenir, le restaurateur d'à côté est en difficulté.

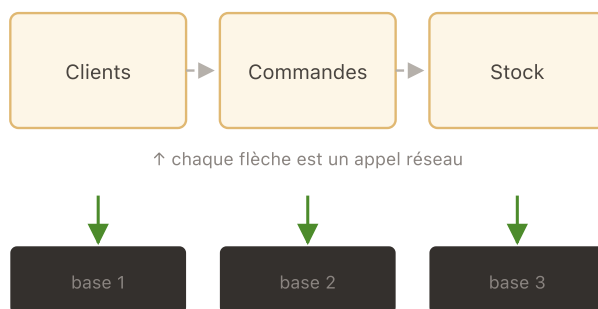
↔ faites glisser

MONOLITHE



1 seul livrable · 1 déploiement · appels de méthodes

MICROSERVICES



N livrables · N déploiements · N supervisions

Une panne n'arrête qu'un service — mais tout passe par le réseau.

Figure 6 — Le même domaine découpé de deux façons. À droite, chaque frontière franchie devient un appel réseau, avec sa latence et ses pannes possibles.

	MONOLITHE	MONOLITHE MODULAIRE	MICROSERVICES
Déploiement	1 seul livrable	1 seul livrable	N livrables indépendants
Découpage interne	Faible ou inexistant	Modules aux frontières nettes	Services séparés physiquement
Base de données	Une seule	Une seule, schémas séparés	Une par service, en principe
Communication	Appels de méthodes	Appels de méthodes	Réseau (HTTP, messages)
Une panne...	...arrête tout	...arrête tout	...n'arrête qu'un service
Difficulté d'exploitation	Faible	Faible	Élevée
Équipe adaptée	1 à 10 personnes	5 à 30 personnes	Plusieurs équipes autonomes

⚠ LES MICROSERVICES NE SONT PAS « LA VERSION MODERNE »

Ce n'est pas un progrès technique, c'est un **arbitrage organisationnel**. On accepte une complexité d'exploitation considérable — réseau, latence, pannes partielles, cohérence des données, supervision — en échange de l'autonomie de plusieurs équipes. Sur un projet à cinq développeurs, ce marché est perdant.

La recommandation courante du métier reste : **commencez par un monolithe modulaire**, et ne découpez que lorsque la douleur devient réelle et identifiée.

6.2 Couches ou tiers ?

Les deux mots se ressemblent, on les confond en permanence, et ils ne parlent pas de la même chose.

COUCHE (LAYER)	TIER
Découpage logique — dans le code	Découpage physique — sur les machines
Des paquets, des dossiers	Des serveurs, des processus
Présentation / métier / persistance	Poste client / serveur applicatif / serveur de base

LA PHRASE QUI RÉSUME

Une application peut très bien être en **trois couches** et déployée sur **un seul tier** : trois paquets Java bien rangés dans un unique fichier installé sur une seule machine. Couches et tiers sont deux axes indépendants.

6.3 Architecture hexagonale



L'ADAPTATEUR DE VOYAGE

Votre ordinateur portable fonctionne en France, au Japon et au Brésil. Vous ne changez pas d'ordinateur : vous changez d'**adaptateur**. L'ordinateur définit ce dont il a besoin ; le monde extérieur s'y branche.

L'**architecture hexagonale**, aussi appelée *ports and adapters* — « ports et adaptateurs » — et proche de la *Clean Architecture*, pousse l'inversion des dépendances à son terme. Le code métier est placé **au centre** et ne dépend de rien. Autour, des adaptateurs : un pour le web, un pour PostgreSQL, un pour l'envoi d'e-mails.

↔ faites glisser

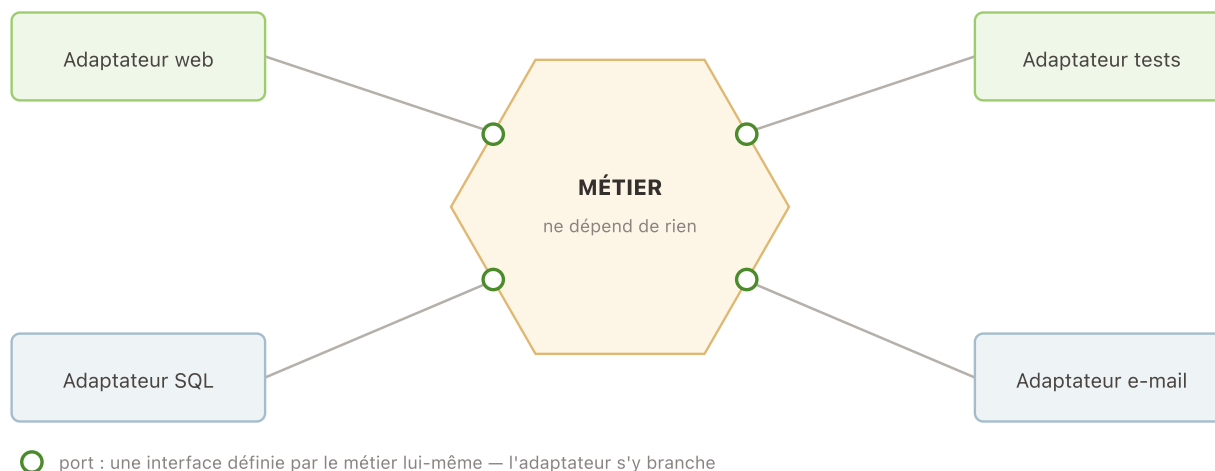


Figure 7 — Remplacer PostgreSQL par un autre stockage revient à écrire un nouvel adaptateur : le centre ne bouge pas.

💡 POUR L'INSTANT, RETENEZ JUSTE L'IDÉE

L'hexagonal est un sujet de deuxième année de pratique. Ce que vous devez en garder aujourd'hui : c'est **le même principe** que le « D » de SOLID vu en 3.4, appliqué à l'application entière au lieu d'une seule classe.

SECTION 07

Qu'est-ce qu'une API ?

Le mot est partout, il est rarement défini. C'est pourtant l'un des plus simples du métier.

7.1 La carte du restaurant



LE RESTAURANT, ENCORE

Vous entrez dans un restaurant. Vous n'allez pas en cuisine choisir vos ingrédients : on vous tend **la carte**.

La carte vous dit trois choses, et seulement trois : **ce que vous pouvez demander** (les plats disponibles), **comment le demander** (« saignant ou à point ? »), et **ce que vous allez recevoir** (une description du plat). Elle ne vous dit pas la recette, ni la marque du four, ni le fournisseur de viande.

C'est exactement une **API** : la carte d'un programme.

API est le sigle de *Application Programming Interface*, en français **interface de programmation applicative**. C'est l'ensemble des points d'entrée qu'un programme **expose** pour que d'autres programmes s'en servent — sans jamais avoir à connaître son fonctionnement interne.

Une API se définit toujours par trois éléments :

ÉLÉMENT DU CONTRAT	AU RESTAURANT	DANS UNE API WEB
Ce qu'on peut demander	La liste des plats	La liste des adresses : <code>/clients</code> , <code>/commandes</code>
Comment le demander	« Cuisson ? Entrée ou plat ? »	Le verbe HTTP, les paramètres, le corps de la requête
Ce qu'on reçoit	La description du plat	Le format de la réponse : JSON, avec tels champs



LA DÉFINITION EN UNE PHRASE

Une API, c'est un **contrat** : ce que je promets d'accepter, et ce que je promets de renvoyer. Tout le reste — le langage, la base de données, les algorithmes — est caché derrière, et peut changer sans prévenir tant que le contrat est respecté.

7.2 Vous en écrivez déjà sans le savoir

RAPPEL DU COURS DE POO JAVA

Quand vous écrivez `public` devant une méthode et `private` devant un attribut, vous êtes **en train de définir une API**. L'ensemble des membres `public` d'une classe est son API : c'est ce que le monde extérieur a le droit d'appeler. Le `private` est la cuisine.

JAVA

```
public class CompteBancaire {  
  
    private double solde;           // LA CUISINE – invisible du dehors  
    private List<Operation> historique; // invisible aussi  
  
    // — L'API de cette classe : ce que les autres peuvent appeler —  
    public void deposer(double montant) { ... }  
    public boolean retirer(double montant) { ... }  
    public double getSolde() { ... }  
}
```

Analysons ce code ligne par ligne :

1. `private double solde` — personne à l'extérieur ne peut modifier le solde directement. On ne rentre pas en cuisine changer le contenu de la casserole.
2. `public void deposer(double montant)` — un point d'entrée du contrat. Le nom, les paramètres et le type de retour **sont** le contrat.
3. `public boolean retirer(...)` — le contrat annonce même son résultat : un booléen, donc « ça peut échouer ». Le code appelant sait à quoi s'attendre sans lire l'intérieur.
4. Demain, vous pouvez réécrire entièrement l'intérieur de `retirer` : tant que la signature ne bouge pas, aucun code appelant ne casse.

Une API web, c'est le même principe, franchi d'un cran : au lieu d'exposer des méthodes à d'autres classes du même programme, on expose des adresses à d'autres **programmes**, à travers le réseau.

7.3 Les trois familles d'API

FAMILLE	QUI EXPOSE	EXEMPLE CONCRET	PASSE PAR LE RÉSEAU ?
API de classe ou de bibliothèque	Une classe, un paquet, une bibliothèque	<code>list.add(...)</code> , <code>Math.random()</code>	Non
API système	Le système d'exploitation	Lire un fichier, ouvrir une fenêtre	Non
API web (service web)	Une application distante	GET <code>https://api.exemple.fr/clients</code>	Oui

⚠ « API » NE VEUT PAS DIRE « SITE WEB »

Une API n'est ni un serveur, ni une base de données, ni un langage. C'est **une porte et son mode d'emploi**. La très grande majorité des API dont on parle aujourd'hui sont des API web, mais l'API de la classe `String` est une API tout aussi authentique.

7.4 À quoi sert une API web

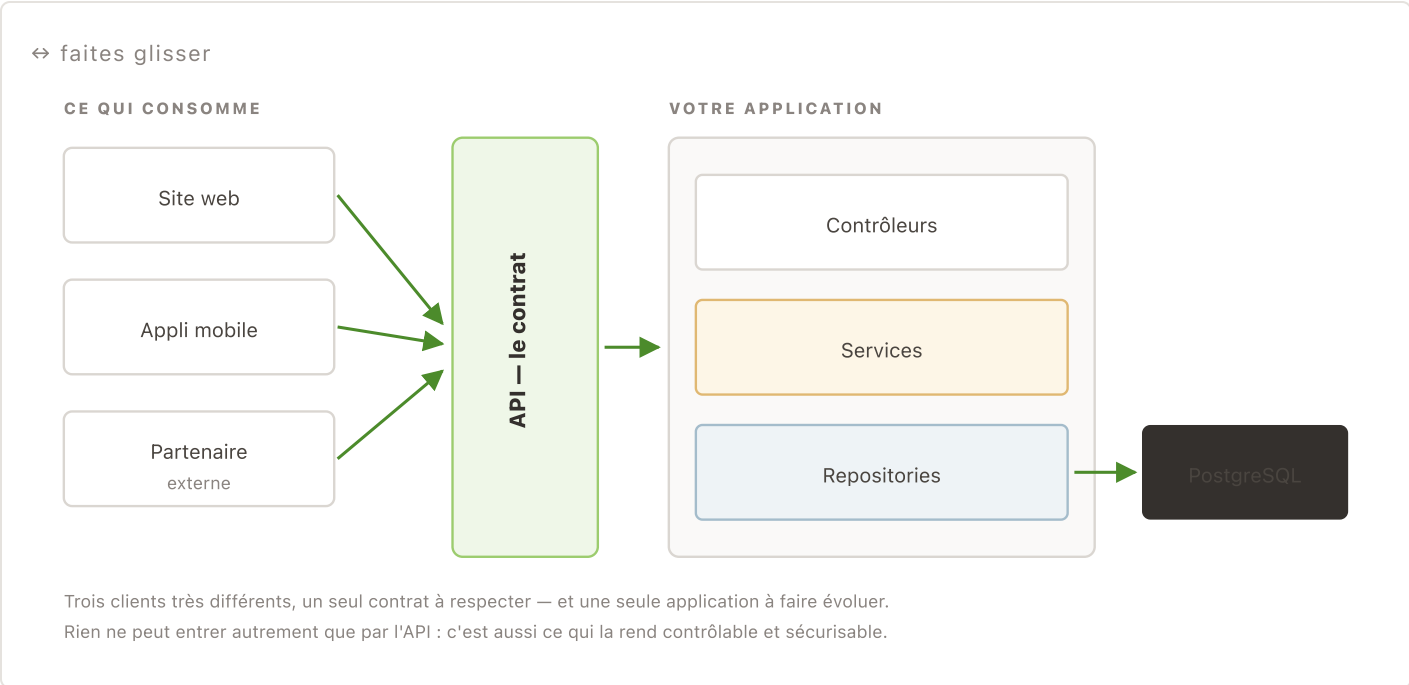


Figure 8 — L'API est la seule porte. Ce qu'il y a derrière peut être entièrement réécrit sans qu'aucun client ne s'en aperçoive.

BÉNÉFICE	CE QUE ÇA CHANGE CONCRÈTEMENT
Un seul métier, plusieurs écrans	Le site web, l'appli mobile et le partenaire consomment la même API. La règle « pas de commande en dessous de 10 € » est écrite une seule fois.
Des équipes en parallèle	Une fois le contrat signé, l'équipe front et l'équipe back travaillent en même temps sans s'attendre.
La liberté technique	Vous pouvez changer PostgreSQL, réécrire un service, migrer de version : tant que le contrat tient, personne dehors ne le voit.
La sécurité	Une seule porte à surveiller, à authentifier et à limiter en débit.
L'ouverture	C'est ce qui permet à un partenaire, ou à un client payant, d'utiliser votre service sans que vous lui livriez votre code.

7.5 Un contrat, ça ne se casse pas

Puisqu'une API est un contrat, le modifier engage tous ceux qui l'utilisent. On appelle **rupture de compatibilité** (*breaking change*) toute modification qui casse le code des clients existants.

MODIFICATION	CASSE LES CLIENTS ?
Ajouter un champ dans la réponse	Non — les anciens l'ignorent
Ajouter une nouvelle adresse	Non
Renommer un champ existant	Oui
Supprimer un champ	Oui
Rendre obligatoire un paramètre facultatif	Oui

💡 D'OÙ LE NUMÉRO DE VERSION DANS LES ADRESSES

C'est la raison pour laquelle vous voyez si souvent `/api/v1/clients`. Quand un changement casse le contrat, on n'écrase pas l'ancien : on publie `/api/v2/clients` à côté, et on laisse aux clients le temps de migrer. Comme un restaurant qui garde l'ancienne carte quelques mois.

SECTION 08

L'API REST

La forme d'API web que vous rencontrerez dans la quasi-totalité des projets.

8.1 Le distributeur automatique



MÉTAPHORE DU DISTRIBUTEUR

Un distributeur automatique ne vous reconnaît pas. Il ne se souvient pas que vous êtes déjà passé ce matin, ni de ce que vous aviez choisi. Chaque achat doit contenir **tout le nécessaire** : la pièce et le code du produit.

C'est une contrainte pour vous. C'est une libération pour l'exploitant : il peut en installer dix identiques dans le hall, sans qu'ils aient besoin de se parler. N'importe lequel peut vous servir.

REST — *REpresentational State Transfer* — est un **style** d'architecture pour les API web, décrit en 2000 par Roy Fielding. Ce n'est ni un protocole, ni une bibliothèque, ni un standard imposé : c'est un ensemble de conventions qui s'appuient sur HTTP.

Quatre principes suffisent pour votre niveau :

PRINCIPE	CE QU'IL IMPOSE
Tout est ressource	Chaque chose manipulable a une adresse : un client, une commande, une facture
Le verbe dit l'action	L'adresse nomme la chose, le verbe HTTP dit ce qu'on lui fait
Sans état (<i>stateless</i>)	Chaque requête se suffit à elle-même. Le serveur ne retient rien entre deux appels
Une représentation	On n'échange pas l'objet, mais une représentation : le plus souvent du JSON

⚠ « REST » EST EMPLOYÉ À TORT ET À TRAVERS

L'immense majorité des API dites « REST » ne respectent qu'une partie des critères de Fielding. Dans le métier, « API REST » signifie en pratique : *API web en HTTP, avec des URL de ressources, des verbes HTTP et du JSON*. C'est ce sens-là que vous utiliserez, et il est admis.

8.2 La ressource et son adresse

Une **ressource** est une chose, désignée par un **nom**, au pluriel, dans l'adresse (*URL*). Jamais un verbe.

ADRESSE	CE QU'ELLE DÉSIGNE
<code>/clients</code>	La collection de tous les clients
<code>/clients/42</code>	Un client précis, celui d'identifiant 42
<code>/clients/42/commandes</code>	Les commandes de ce client — la jointure, vue de l'extérieur
<code>/clients?</code> <code>ville=Lyon&page=2</code>	La collection filtrée et paginée : les filtres passent en paramètres, pas dans le chemin

⚠ UN VERBE DANS L'URL EST UNE FAUTE

On n'écrit pas `GET /supprimerClient?id=42`, ni `POST /creerClient`. On écrit `DELETE /clients/42` et `POST /clients`. Le verbe est déjà porté par HTTP : le répéter dans l'adresse est le signe d'une API mal conçue, et c'est la première chose qu'un correcteur regarde.

💡 TROIS CONVENTIONS QUI FONT SÉRIEUX

Pluriel pour les collections (`/clients`, pas `/client`) · minuscules et tirets (`/bons-de-commande`) · numéro de version en tête (`/api/v1/...`).

8.3 Les verbes HTTP, CRUD et SQL

Voici le tableau qui relie tout ce que vous savez déjà. **CRUD** est l'acronyme des quatre opérations élémentaires sur une donnée : *Create, Read, Update, Delete* — créer, lire, modifier, supprimer.

VERBE HTTP	ADRESSE	INTENTION	CRUD	SQL	STATUT ATTENDU
GET	/clients	Lister	Read	SELECT	200
GET	/clients/42	Lire un client	Read	SELECT ... WHERE id = 42	200 ou 404
POST	/clients	Créer	Create	INSERT	201
PUT	/clients/42	Remplacer entièrement	Update	UPDATE	200 ou 204
PATCH	/clients/42	Modifier partiellement	Update	UPDATE (colonnes ciblées)	200
DELETE	/clients/42	Supprimer	Delete	DELETE	204

⚠ POST ET PUT NE SONT PAS INTERCHANGEABLES

POST /clients **crée** : appelé trois fois, il crée trois clients. PUT /clients/42 **remplace** : appelé trois fois, le résultat est identique à un seul appel. Cette propriété s'appelle l'**idempotence** — « le même effet, quel que soit le nombre d'appels ». GET, PUT et DELETE sont idempotents ; POST ne l'est pas.

8.4 Anatomie d'une requête et d'une réponse



Figure 9 — Une requête et une réponse HTTP ont la même structure : une première ligne, des en-têtes, un corps facultatif.

Analysons cette requête ligne par ligne :

1. `POST /api/v1/clients` — le **verbe** et la **ressource**. À eux deux, ils disent tout : « ajoute un élément à la collection des clients ».
2. `Content-Type: application/json` — un **en-tête**. Il annonce le format du corps envoyé, pour que le serveur sache le décoder.
3. `Authorization: Bearer ...` — le jeton d'authentification. Il est renvoyé **à chaque requête**, puisque le serveur ne se souvient de rien.
4. Le **corps** (*body*) — les données envoyées, en JSON. `GET` et `DELETE` n'en ont pas ; `POST`, `PUT` et `PATCH` en ont un.
5. En réponse, `201 Created` annonce la création, et l'en-tête `Location` indique où trouver la ressource nouvellement créée.

8.5 Les codes de statut



LES TROIS ÉTAGES DU CODE

Le premier chiffre suffit à comprendre : **2** = c'est bon, **4** = c'est de votre faute, **5** = c'est de la nôtre.

CODE	NOM	QUAND L'UTILISER
200	OK	Lecture ou modification réussie, avec un corps en réponse
201	Created	Création réussie — la réponse à un <code>POST</code>
204	No Content	Réussi, mais rien à renvoyer — typiquement un <code>DELETE</code>
400	Bad Request	La requête est mal formée ou les données sont invalides
401	Unauthorized	Vous n'êtes pas identifié
403	Forbidden	Vous êtes identifié, mais vous n'avez pas le droit
404	Not Found	La ressource demandée n'existe pas
409	Conflict	Conflit avec l'état actuel — un e-mail déjà utilisé, par exemple
500	Internal Server Error	Une erreur non prévue dans votre code

⚠ LE PIÈGE DU « 200 QUI CONTIENT UNE ERREUR »

Renvoyer `200 OK` avec un corps `{ "erreur": "client introuvable" }` est une faute courante et pénible. Le client automatisé, lui, ne lit que le code : il croira que tout va bien. Le code de statut **fait partie du contrat**, au même titre que le JSON.

💡 401 OU 403 ?

Une phrase pour ne plus hésiter : **401, je ne sais pas qui vous êtes ; 403, je sais qui vous êtes et c'est non.**

8.6 JSON, le format d'échange

JSON (*JavaScript Object Notation*) est le format de texte utilisé pour transporter les données. Il est lisible par un humain et par tous les langages.

JSON

```
// La représentation JSON d'un client
{
  "id": 42,
  "nom": "Dupont",
  "email": "d@ex.fr",
  "dateInscription": "2026-03-14",
  "commandes": [1001, 1002]
}
```

Ce texte n'est pas un objet Java : c'est sa **représentation**. Le passage de l'un à l'autre porte deux noms que vous croiserez souvent : la **sérialisation** (objet → texte) et la **désérialisation** (texte → objet). Le framework s'en charge.

8.7 Sans état : la conséquence pratique

Le serveur ne garde rien entre deux requêtes. Cela a une conséquence directe sur l'authentification : il n'y a pas de « session ouverte » côté serveur. Le client renvoie son **jeton** (*token*) à chaque appel, dans l'en-tête `Authorization`.

🎯 POURQUOI C'EST UN AVANTAGE

Parce qu'aucun serveur n'est propriétaire de votre conversation, on peut en démarrer dix identiques derrière un répartiteur de charge. N'importe lequel peut traiter n'importe quelle requête — comme n'importe quel distributeur du hall.

SECTION 09

Le flux CRUD de bout en bout

Le schéma le plus important de la séance : ce qui se passe entre le clic de l'utilisateur et la ligne dans PostgreSQL.

Vous avez maintenant toutes les pièces. Assemblons-les. Les quatre opérations CRUD suivent **exactement le même trajet** : elles entrent par l'API, traversent les trois couches et finissent en SQL.

9.1 Le grand schéma

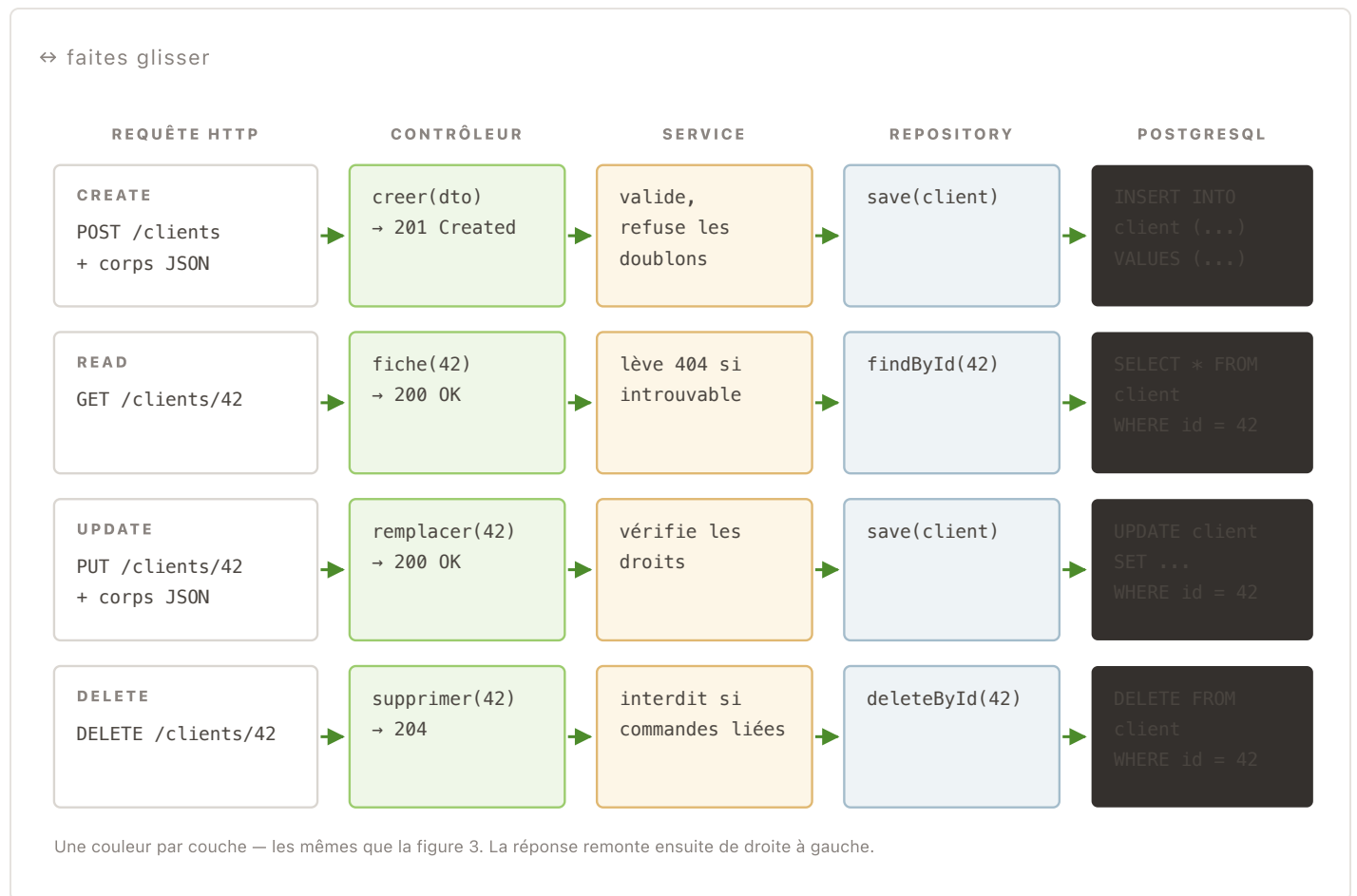


Figure 10 — Les quatre opérations CRUD de bout en bout. Chaque ligne se lit de gauche à droite ; les couleurs reprennent celles des couches.

🎯 CE QUE CE SCHÉMA VOUS DIT

Quatre opérations, un seul trajet. Quand vous saurez écrire **une** ligne de ce tableau, vous saurez écrire les trois autres : seuls le verbe, le nom de la méthode et la requête SQL changent. C'est ce qui rend une API CRUD si rapide à produire avec un framework.

9.2 Le parcours de la donnée

La donnée ne traverse pas ces couches sans changer de forme. Elle est **transformée quatre fois**, et chaque transformation a un nom que vous entendrez.

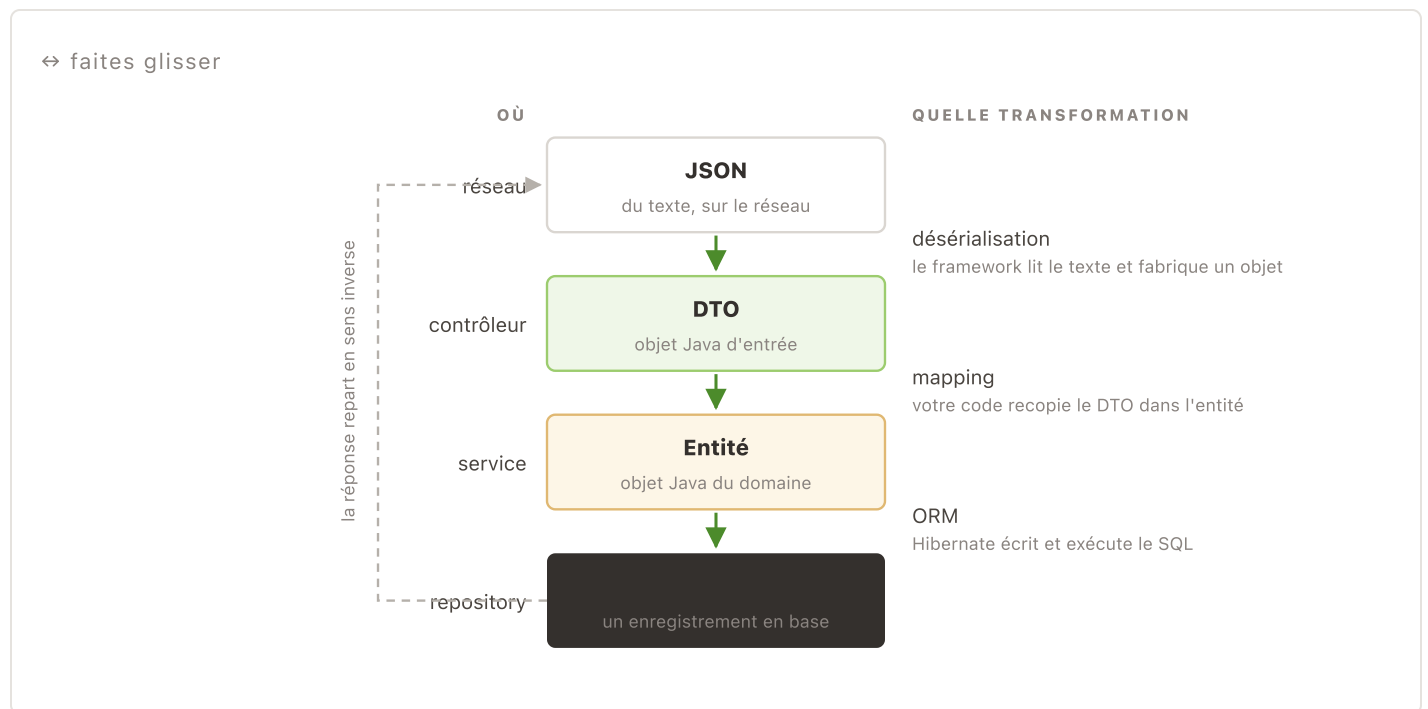


Figure 11 — La même information, quatre représentations. Confondre ces quatre formes est la source d'erreur la plus fréquente des débutants.

⚠ QUATRE FORMES, QUATRE NOMS — NE LES MÉLANGEZ PAS

Le **JSON** est du texte. Le **DTO** est un objet fait pour entrer et sortir. L'**entité** est un objet fait pour le domaine et la base. La **ligne** est ce qui existe réellement dans PostgreSQL. Utiliser une entité comme DTO, c'est ouvrir sa réserve aux clients ; utiliser un DTO comme entité, c'est confier ses règles métier à la salle.

9.3 Une création, pas à pas

Suivons une seule requête, `POST /clients`, du début à la fin.

```

// ÉTAPE 1-2 – le framework reçoit le texte et fabrique un DTO
// { "nom": "Dupont", "email": "d@ex.fr" } → CreerClientDto

// ÉTAPE 3 – le CONTRÔLEUR reçoit l'objet, ne décide rien
@PostMapping("/clients")
public ResponseEntity<ClientDto> creer(@RequestBody CreerClientDto dto) {
    Client cree = service.creer(dto.nom(), dto.email()); // il délègue
    return ResponseEntity
        .status(201) // le bon code
        .body(ClientDto.depuis(cree)); // un DTO, pas l'entité
}

// ÉTAPE 4 – le SERVICE décide
public Client creer(String nom, String email) {
    if (repository.existsByEmail(email)) { // règle métier
        throw new EmailDejaUtilise(email); // → deviendra un 409
    }
    Client client = new Client(nom, email); // l'entité
    return repository.save(client); // il délègue à son tour
}

// ÉTAPE 5-6 – le REPOSITORY et l'ORM produisent le SQL
// INSERT INTO client (nom, email, date_inscription) VALUES (?, ?, ?)
// → PostgreSQL attribue l'id 42

// ÉTAPE 7 – la réponse repart : entité → DTO → JSON
// 201 Created + { "id": 42, "nom": "Dupont", "email": "d@ex.fr" }

```

Analysons ce parcours étape par étape :

1. **Désérialisation** — le framework transforme le texte JSON en `CreerClientDto`. Vous n'écrivez pas cette ligne : `@RequestBody` la déclenche.
2. **Contrôleur** — il ne teste rien, ne calcule rien. Il appelle le service et choisit le code de statut. Trois lignes, c'est normal.
3. **Service** — c'est ici, et nulle part ailleurs, que vit la règle « un e-mail ne peut pas servir deux fois ». Elle est testable sans base et sans réseau.
4. **Repository** — `save()` ne contient aucune décision. L'ORM traduit en `INSERT`.
5. **Retour** — l'entité repart en DTO, puis en JSON. L'entité elle-même ne sort jamais de l'application.

💡 LE RÉFLEXE DE PLACEMENT

Devant n'importe quelle ligne de code à écrire, posez-vous une seule question : « **est-ce que c'est une décision ?** » Si oui, elle va dans le service. Si c'est de la traduction (JSON, codes HTTP, formats), elle va dans le contrôleur. Si c'est du rangement, elle va dans le repository.

9.4 Où placer quoi — le tableau de tri

CE QUE VOUS DEVEZ ÉCRIRE	COUCHE	POURQUOI
Choisir de renvoyer 201 plutôt que 200	Contrôleur	C'est du protocole HTTP, pas du métier
Refuser un e-mail en double	Service	C'est une règle du domaine
Masquer le mot de passe dans la réponse	Contrôleur (via le DTO)	C'est une décision d'exposition
Écrire <code>WHERE actif = true</code>	Repository	C'est de l'accès aux données
Calculer une remise de 10 % au-delà de 100 €	Service	Règle métier, réutilisable par le web et le mobile
Convertir une date en texte pour l'écran	Contrôleur / DTO	C'est de la présentation
Ouvrir et fermer une transaction	Service	Une transaction couvre une opération métier complète

Deux ponts vers Spring Boot

Deux mécanismes qui relient tout ce qui précède à ce que vous ouvrirez la semaine prochaine.

10.1 L'injection de dépendances



ON REBRANCHE LA PRISE

Vous ne produisez pas votre électricité : vous branchez. Mieux, dans une maison neuve, vous ne posez pas non plus les câbles — l'installation existe, vous n'avez qu'à brancher.

Le **conteneur IoC** de Spring, c'est cette installation électrique : il fabrique les objets et les branche les uns aux autres à votre place.

En 3.4, nous avons fait entrer la dépendance par le constructeur. Il restait un problème : **quelqu'un** doit bien écrire le `new` quelque part.

JAVA

```
// AVANT : quelqu'un doit tout assembler à la main
ClientDao dao = new ClientDaoPostgres(connexion);
ClientService service = new ClientService(dao);
ClientController ctrl = new ClientController(service);
// ... et ainsi de suite sur 300 classes
```

JAVA

```
// APRÈS : le conteneur assemble seul
@Service                                     // « je suis un objet à gérer »
public class ClientService {

    private final ClientDao dao;

    public ClientService(ClientDao dao) { // « il me faut un ClientDao »
        this.dao = dao;                 // Spring le trouve et le fournit
    }
}
```

Analysons ce code ligne par ligne :

1. `@Service` — l'étiquette déclare la classe au conteneur. Au démarrage, Spring parcourt le projet, repère l'étiquette et fabrique **un** exemplaire de la classe.
2. `ClientService(ClientDao dao)` — le constructeur exprime un besoin : « donnez-moi quelque chose qui respecte le contrat `ClientDao` ».
3. Spring cherche dans son inventaire une classe qui implémente ce contrat, et la passe automatiquement. Vous n'écrivez plus aucun `new`.

C'est l'inversion de contrôle de la section 2, appliquée non plus au déroulement du programme mais à la **construction des objets**.

INJECTEZ TOUJOURS PAR LE CONSTRUCTEUR

Spring propose trois voies : constructeur, setter, ou directement sur l'attribut avec `@Autowired`. Préférez systématiquement le **constructeur** : il rend les dépendances visibles, autorise le `final`, et permet d'instancier la classe à la main dans un test.

10.2 L'ORM



L'INTERPRÈTE

Deux personnes doivent travailler ensemble. L'une parle le langage des tableaux : lignes, colonnes, jointures. L'autre parle le langage des objets : classes, attributs, références. Elles ne se comprennent pas. On engage un **interprète**, qui traduit dans les deux sens sans jamais participer à la conversation.

Un **ORM** (*Object-Relational Mapping* — « correspondance objet-relationnel ») est cet interprète. En Java, la norme s'appelle **JPA** et l'implémentation la plus répandue **Hibernate**.

CE QUE VOUS ÉCRIVEZ	CE QUE L'ORM PRODUIT
<code>repository.findById(42L)</code>	<code>SELECT * FROM client WHERE id = 42</code>
<code>repository.save(client)</code>	<code>INSERT</code> ou <code>UPDATE</code> , selon le cas
<code>client.getCommandes()</code>	<code>SELECT * FROM commande WHERE id_client = 42</code>

La traduction n'est pas parfaite, parce que les deux mondes ne sont pas superposables. On appelle cet écart le **décalage objet-relationnel** (*impedance mismatch*) :

EN JAVA	EN BASE RELATIONNELLE	LE PROBLÈME
L'héritage existe	Il n'existe pas	Il faut choisir une stratégie de traduction
Deux objets peuvent être égaux sans être le même	La clé primaire tranche	Deux notions d'identité différentes
Une référence est à sens unique	Une jointure se lit dans les deux sens	Il faut désigner un côté « propriétaire »
Les collections sont chargées en mémoire	Une table peut faire dix millions de lignes	Quand charge-t-on ? D'où le chargement paresseux

⚠ L'ORM NE VOUS DISPENSE PAS DE SAVOIR ÉCRIRE DU SQL

Il le rend au contraire **plus nécessaire**. Un ORM mal utilisé transforme une boucle sur 100 clients en 101 requêtes — c'est le fameux problème « N+1 ». Sans lire le SQL généré, vous ne verrez jamais le problème ; vous constaterez seulement que l'application est lente. Vos séances PostgreSQL vont servir plus que vous ne le pensez.

Où se place Spring Boot

Tout ce qui précède se recompose en une seule image.



DERNIER PASSAGE SUR LE CHANTIER

Spring Framework, c'est le constructeur de maisons préfabriquées : il fournit la structure porteuse — le conteneur IoC — et un catalogue d'options. Historiquement, il fallait remplir un long formulaire de configuration pour choisir chaque élément.

Spring Boot, c'est le même constructeur avec un catalogue « clés en main ». Vous cochez *maison familiale*, il livre chauffage, électricité et cuisine équipée, déjà branchés, aux emplacements standards. Vous restez libre de tout changer — mais si les emplacements standards vous conviennent, vous n'avez rien à faire.

↔ faites glisser

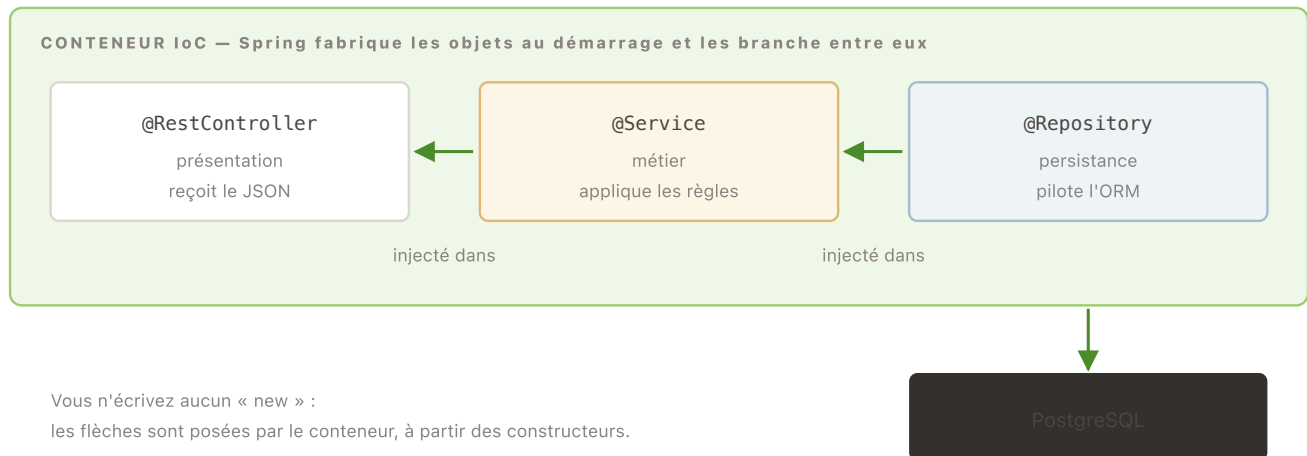


Figure 12 — Les étiquettes ne font rien par elles-mêmes : elles disent au conteneur quoi fabriquer et où le brancher.

NOTION SPRING BOOT	CE QU'ELLE SIGNIFIE	VU EN SECTION
Conteneur IoC	Il fabrique vos objets et les branche entre eux	2.2 et 10.1
Convention plutôt que configuration	Si vous suivez les usages, vous ne configurez rien	2.3
Auto-configuration	Il détecte les bibliothèques présentes et se règle seul	2.2
<i>Starters</i>	Des lots de dépendances cohérentes, testées ensemble	2.3
Serveur embarqué	Le serveur web est dans l'application : un seul fichier à lancer	6.2
Spring MVC	Le module qui applique le patron MVC côté serveur	5.1
Spring Web / REST	Le module qui expose votre API en HTTP et JSON	7 et 8
Spring Data JPA	Le module qui pilote l'ORM et fabrique vos repositories	10.2

LA BONNE NOUVELLE

Vous connaissez déjà les deux extrémités de ce schéma : PostgreSQL en bas, les classes Java partout. Il ne vous reste qu'à apprendre les étiquettes qui relient le tout — et à savoir **dans quelle boîte poser chaque ligne de code**. C'était l'objet de cette séance.

Ce qu'il faut retenir

NOTION	EN UNE PHRASE	L'IMAGE
Bibliothèque	Vous l'appellez quand vous voulez	La boîte à outils
Framework	C'est lui qui appelle votre code	La maison préfabriquée
Inversion de contrôle (IoC)	Le contrôle du déroulement passe au framework	« Ne nous appelez pas, on vous appellera »
Injection de dépendances (DI)	Une <i>technique</i> d'IoC : on vous fournit vos outils	La prise électrique
Séparation des responsabilités	Une partie du code = un seul métier	Le serveur ne cuisine pas
Couplage / cohésion	Couplage faible, cohésion forte	La guirlande / le tiroir à couverts
Architecture en couches	Présentation → métier → persistance, sans saut	Salle → cuisine → réserve
Entité / DTO	Le reflet de la table / ce qu'on accepte de montrer	Le carton en réserve / l'assiette dressée
MVC	Le contrôleur fait le lien, la vue lit le modèle	Le serveur porte l'assiette
MVP	Le présentateur dicte tout, la vue est passive	Le serveur dresse à table
MVVM	La vue se lie au ViewModel, tout se rafraîchit seul	Le passe-plat automatique
Monolithe / microservices	Un seul livrable / N livrables indépendants	Le grand magasin / la rue commerçante

NOTION	EN UNE PHRASE	L'IMAGE
Couche / tier	Découpage du code / découpage des machines	Les rayons / les bâtiments
Hexagonale	Le métier au centre, des adaptateurs autour	L'adaptateur de voyage
API	Un contrat : ce que j'accepte, ce que je renvoie	La carte du restaurant
API REST	Une adresse = une ressource, un verbe = une action, sans état	Le distributeur automatique
CRUD	Créer, lire, modifier, supprimer — les quatre opérations de base	POST, GET, PUT, DELETE
Codes de statut	2 = c'est bon, 4 = votre faute, 5 = la nôtre	Les trois étages
ORM	Traduit les tables en objets et retour	L'interprète
Spring Boot	Spring, préconfiguré et prêt à démarrer	La maison clés en main

SECTION 13

Réflexes et pièges

Les confusions qui reviennent en évaluation et en entretien d'embauche.

	L'AFFIRMATION FAUSSE	CE QU'IL FAUT RÉPONDRE
1	« Un framework est une grosse bibliothèque »	Le critère est le sens de l'appel, pas la taille
2	« IoC et injection de dépendances, c'est pareil »	L'IoC est le principe, la DI une technique parmi d'autres
3	« MVC remplace l'architecture en couches »	MVC organise la présentation ; les couches organisent l'application. Les deux coexistent
4	« Le M de MVC, c'est la base de données »	C'est le domaine : les données et les règles
5	« MVVM, c'est pour le desktop »	Le vrai critère est la présence d'une liaison de données automatique
6	« Une couche et un tier, c'est la même chose »	Logique contre physique. Trois couches peuvent tenir sur une machine
7	« Les microservices, c'est mieux »	C'est un arbitrage organisationnel très coûteux à exploiter
8	« Une API, c'est un site web »	C'est un contrat. Les méthodes <code>public</code> d'une classe Java en sont une aussi
9	« <code>GET /supprimerClient?id=42</code> »	L'adresse nomme une ressource, le verbe dit l'action : <code>DELETE /clients/42</code>
10	« POST et PUT, c'est pareil »	POST crée à chaque appel ; PUT est idempotent
11	« On renvoie 200 avec un message d'erreur »	Le code de statut fait partie du contrat : 404, 409, 400...
12	« REST garde ma session ouverte »	REST est sans état : le jeton repart à chaque requête
13	« On peut renvoyer l'entité directement »	C'est la fuite de données classique. On passe par un DTO

	L'AFFIRMATION FAUSSE	CE QU'IL FAUT RÉPONDRE
14	« Avec un ORM, plus besoin de SQL »	Il en faut davantage : voir le problème N+1

LES QUATRE PHRASES À SAVOIR RÉCITER

1. Une bibliothèque, je l'appelle ; un framework, il m'appelle.
2. Couplage faible, cohésion forte.
3. Chaque couche ne parle qu'à celle du dessous.
4. Une API est un contrat : l'adresse nomme la chose, le verbe dit l'action.

AVANT LA PROCHAINE SÉANCE

Reprenez un MCD que vous avez déjà produit. Sans écrire une ligne de code, dites à voix haute, pour chacune de ses entités : quelle serait la table, quelle serait la classe entité, quel serait le DTO, quelles règles métier iraient dans le service, et quelles seraient les quatre adresses REST de son CRUD. Si vous savez faire cela, Spring Boot ne sera plus qu'une affaire de syntaxe.